

Informe de Práctica: Diseño y Fragmentación de una Base de Datos Distribuida

Estudiante: Freddy Vladimir
Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software
Asignatura: Aplicaciones Distribuidas ISR-701

Período Regular 2026–2027

1. Introducción

La presente práctica tiene como finalidad diseñar e implementar una base de datos distribuida mediante técnicas de fragmentación horizontal, vertical y mixta. Para ello se utilizó PostgreSQL 16 dentro de contenedores Docker, simulando tres nodos independientes que representan las sedes Campus, Babahoyo y Ventanas.

La fragmentación de bases de datos permite dividir la información en partes más pequeñas, llamadas fragmentos, que pueden almacenarse en diferentes nodos. Esta estrategia mejora la organización de los datos, permite acercar la información al lugar donde más se utiliza y facilita el crecimiento de un sistema distribuido [1, 2]. En esta práctica se trabajó con el escenario de una cafetería universitaria que registra clientes, productos y pedidos.

El objetivo principal fue comprobar que una base de datos puede ser dividida físicamente en varios nodos y, posteriormente, reconstruida mediante vistas globales usando `postgres_fdw`. Además, se verificaron las tres condiciones fundamentales de una fragmentación correcta: completitud, reconstrucción y disjunción.

2. Entorno de trabajo

Para el desarrollo de la práctica se utilizó un entorno local basado en Docker Desktop, PostgreSQL 16 y PowerShell. El proyecto fue gestionado desde IntelliJ IDEA, donde se organizaron los archivos SQL, el archivo `docker-compose.yml`, el archivo `README.md` y el informe en $\text{\LaTeX}$.

La estructura principal del proyecto fue la siguiente:

```
1 practica-fragmentacion/  
2 |  
3 |-- docker-compose.yml  
4 |-- README.md  
5 |-- sql/
```

```

6 |      |-- 01_esquema_central.sql
7 |      |-- 02_datos.sql
8 |      |-- 03_fragmentacion_horizontal.sql
9 |      |-- 04_fragmentacion_vertical.sql
10 |     |-- 05_fragmentacion_mixta.sql
11 |     |-- 06_vistas_globales.sql
12 |     |-- 07_verificacion.sql
13 |
14 |-- informe/
15 |     |-- informe.tex
16 |     |-- capturas/

```

Se levantaron tres contenedores PostgreSQL. Cada contenedor representa un nodo de la base de datos distribuida.

Cuadro 1: Nodos utilizados en la práctica

Nodo	Contenedor	Puerto local	Base de datos
Campus	pg-campus	5433	cafeteria
Babahoyo	pg-babahoyo	5434	cafeteria
Ventanas	pg-ventanas	5435	cafeteria

Los tres contenedores fueron levantados con el comando:

```
1 docker compose up -d
```

Posteriormente se verificó su estado con:

```
1 docker compose ps
```

3. Modelo de datos centralizado

Antes de realizar la fragmentación, se creó un esquema centralizado en el nodo **pg-campus**. Este esquema sirvió como referencia para comparar los resultados después de dividir los datos.

El modelo utilizado contiene tres tablas principales: **clientes**, **productos** y **pedidos**. La tabla **clientes** almacena la información de los usuarios de la cafetería, la tabla **productos** contiene los productos disponibles y la tabla **pedidos** registra las ventas realizadas.

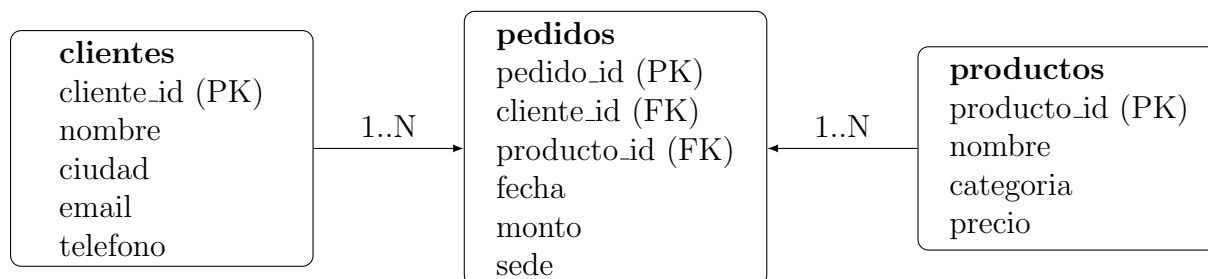


Figura 1: Modelo entidad-relación adaptado a la práctica de Cafetería UTEQ.

Luego se insertaron datos de prueba: 6 clientes, 5 productos y 8 pedidos. Estos datos fueron necesarios para comprobar la fragmentación en los tres nodos.

4. Fragmentación horizontal

La fragmentación horizontal consiste en dividir una tabla por filas. En esta práctica se aplicó sobre la tabla **pedidos**, utilizando como criterio la columna **sede**. De esta manera, cada nodo almacena únicamente los pedidos correspondientes a su sede.

Cuadro 2: Distribución horizontal de pedidos

Nodo	Criterio	Pedidos almacenados
pg-campus	sede = Campus	1, 3, 7
pg-babahoyo	sede = Babahoyo	2, 6, 8
pg-ventanas	sede = Ventanas	4, 5

Este tipo de fragmentación es adecuado porque los pedidos se generan en una sede específica. Por lo tanto, cada sede puede consultar sus propios pedidos sin depender directamente de las demás. Además, los predicados utilizados son excluyentes, ya que un pedido solo pertenece a una sede.

5. Fragmentación vertical

La fragmentación vertical consiste en dividir una tabla por columnas. En este caso se aplicó sobre la tabla **clientes**. Se separaron los datos públicos de los datos de contacto.

Cuadro 3: Fragmentación vertical de clientes

Fragmento	Columnas	Nodo
clientes_publicos	cliente_id, nombre, ciudad	pg-campus
clientes_contacto	cliente_id, email, telefono	pg-babahoyo

La clave primaria **cliente_id** se mantuvo en ambos fragmentos. Esto es necesario porque permite reconstruir la tabla original mediante una operación **JOIN**. Si la clave primaria no se replicara, no sería posible saber a qué cliente pertenece cada correo o teléfono [3].

6. Fragmentación mixta

La fragmentación mixta combina la fragmentación horizontal y vertical. En esta práctica se aplicó sobre la información de clientes. Primero se dividieron los clientes por ciudad y después se separaron los datos públicos de los datos de contacto.

Cuadro 4: Fragmentación mixta aplicada a clientes

Corte horizontal	Corte vertical	Contenido	Nodo
Quevedo	Datos públicos	cliente_id, nombre, ciudad	pg-campus
Quevedo	Datos contacto	cliente_id, email, telefono	pg-babahoyo
Babahoyo/Ventanas	Datos públicos	cliente_id, nombre, ciudad	pg-ventanas
Babahoyo/Ventanas	Datos contacto	cliente_id, email, telefono	pg-babahoyo

Este diseño permite separar la información por ubicación y, al mismo tiempo, proteger los datos sensibles. Los datos públicos se distribuyen según la ciudad, mientras que los datos de contacto se concentran en el nodo Babahoyo.

7. Vistas globales con postgres_fdw

Para reconstruir los datos distribuidos se utilizó la extensión `postgres_fdw`, incluida en PostgreSQL. Esta extensión permite que un nodo consulte tablas ubicadas en otros nodos como si fueran tablas locales, de manera similar a como se reconstruyen fragmentos distribuidos en algoritmos de fragmentación multitarea [4].

Desde el nodo `pg-campus` se crearon servidores remotos hacia `pg-babahoyo` y `pg-ventanas`. Luego se definieron tablas foráneas para acceder a los fragmentos remotos de pedidos y clientes.

La vista `pedidos_global` reconstruyó los pedidos mediante `UNION ALL`:

```

1 CREATE VIEW pedidos_global AS
2 SELECT * FROM pedidos
3 UNION ALL
4 SELECT * FROM pedidos_babahoyo
5 UNION ALL
6 SELECT * FROM pedidos_ventanas;
```

La vista `clientes_global` reconstruyó los clientes mediante `JOIN`:

```

1 CREATE VIEW clientes_global AS
2 SELECT
3     p.cliente_id,
4     p.nombre,
5     p.ciudad,
6     c.email,
7     c.telefono
8 FROM clientes_publicos p
9 JOIN clientes_contacto_remota c
10 ON p.cliente_id = c.cliente_id;
```

8. Verificación de resultados

Para validar la fragmentación se comprobaron las condiciones de completitud, reconstrucción y disjunción.

8.1. Completitud

La completitud indica que ningún dato debe perderse. En la tabla central existían 8 pedidos y en la vista global también se obtuvieron 8 pedidos.

Cuadro 5: Verificación de completitud en pedidos

Origen	Total de filas
pedidos_central	8
pedidos_global	8

8.2. Reconstrucción

La reconstrucción indica que la tabla original puede volver a obtenerse desde sus fragmentos. Para comprobarlo se comparó la suma de ventas por sede entre la tabla central y la vista global.

Cuadro 6: Verificación de reconstrucción por sede

Origen	Sede	Total
central	Babahoyo	4.25
global	Babahoyo	4.25
central	Campus	4.25
global	Campus	4.25
central	Ventanas	2.25
global	Ventanas	2.25

8.3. Disjunción

La disjunción indica que un dato no debe aparecer repetido en más de un fragmento. La consulta de verificación devolvió cero filas, lo que demuestra que ningún pedido fue duplicado.

```
1 SELECT pedido_id, COUNT(*) AS veces
2 FROM pedidos_global
3 GROUP BY pedido_id
4 HAVING COUNT(*) > 1;
```

El resultado fue:

```
1 (0 rows)
```

También se verificó la reconstrucción vertical de clientes. La tabla central contenía 6 clientes y la vista global también devolvió 6 clientes completos.

Cuadro 7: Verificación de completitud en clientes

Origen	Total de filas
clientes_central	6
clientes_global	6

9. Capturas de ejecución

En esta sección se deben insertar las capturas tomadas durante la ejecución de la práctica. Las capturas recomendadas son: estado de los contenedores, fragmentación horizontal, vistas globales y verificación final.

Insertar captura: docker compose ps mostrando los tres contenedores activos.

Figura 2: Contenedores PostgreSQL levantados correctamente.

Insertar captura: consulta `SELECT * FROM pedidos-global ORDER BY pedido_id.`

Figura 3: Reconstrucción global de pedidos.

Insertar captura: consulta `SELECT * FROM clientes-global ORDER BY cliente_id.`

Figura 4: Reconstrucción global de clientes.

Insertar captura: resultado del script `07_verificacion.sql`.

Figura 5: Verificación de completitud, reconstrucción y disjunción.

10. Reflexión

Durante el desarrollo de la práctica se pudo comprender de manera más clara cómo funciona una base de datos distribuida. Al inicio, la información estaba concentrada en un solo nodo, pero después se dividió en diferentes contenedores, simulando sedes reales de una organización. Esto permitió observar que la fragmentación no consiste solamente en separar datos, sino en hacerlo de forma ordenada y comprobable.

La fragmentación horizontal resultó sencilla de entender porque la tabla `pedidos` tenía una columna llamada `sede`. Esta columna permitió enviar cada pedido al nodo correspondiente. En un caso real, esto sería útil porque cada sede podría trabajar principalmente con sus propios datos, reduciendo la carga sobre un servidor central.

La fragmentación vertical permitió observar una ventaja relacionada con la seguridad. Al separar los datos públicos de los datos de contacto, se puede controlar mejor qué usuario o nodo tiene acceso a cierta información. Por ejemplo, un área administrativa podría acceder a correos y teléfonos, mientras que otros usuarios solo verían nombres y ciudades.

La fragmentación mixta fue la parte más compleja porque combina dos criterios al mismo tiempo. Primero se separaron los clientes por ciudad y luego se dividieron sus columnas según el tipo de información. Esta técnica requiere más cuidado porque es fácil olvidar un fragmento o duplicar datos sin darse cuenta.

Una de las principales dificultades fue coordinar correctamente los nodos y los scripts SQL. También fue necesario verificar que cada tabla estuviera en el contenedor correcto.

El uso de `postgres_fdw` fue importante porque permitió reconstruir los datos desde el nodo Campus, consultando información remota de Babahoyo y Ventanas.

En conclusión, la práctica permitió aplicar conceptos fundamentales de bases de datos distribuidas de forma práctica. Se comprobó que la fragmentación debe cumplir completitud, reconstrucción y disjunción. Además, se entendió que un diseño distribuido puede mejorar la organización y escalabilidad, pero también aumenta la complejidad del sistema.

11. Preguntas de análisis

1. Si se abre una nueva sede en El Empalme, ¿qué debe cambiar en el diseño horizontal?

Se debe agregar un nuevo nodo o contenedor para la sede El Empalme y crear un nuevo fragmento horizontal para los pedidos cuya sede sea El Empalme. También habría que actualizar la vista `pedidos_global` para incluir el nuevo fragmento mediante otro `UNION ALL`.

2. ¿En qué situaciones la fragmentación vertical mejora la seguridad?

Mejora la seguridad cuando una tabla contiene datos sensibles que no todos los usuarios deben ver. Por ejemplo, en una tabla de estudiantes se podrían separar los datos públicos, como nombre y carrera, de datos privados como dirección, teléfono o información médica.

3. ¿Qué ventaja da que la clave primaria se replique en la fragmentación vertical?

La clave primaria permite unir nuevamente los fragmentos mediante `JOIN`. Si no se replicara, no existiría una forma segura de relacionar los datos públicos con los datos privados del mismo registro.

4. ¿Por qué la fragmentación horizontal por sede es razonable para pedidos?

Es razonable porque cada pedido pertenece a una sede específica. De esta manera, cada nodo almacena los pedidos que realmente necesita consultar. Fragmentar por producto no sería tan conveniente porque un mismo producto puede venderse en varias sedes.

5. Si se tuviera que soportar tolerancia a fallos, ¿qué se agregaría al diseño?

Se agregaría replicación de datos. Esto permitiría que si un nodo falla, otro nodo tenga una copia de la información y el sistema pueda seguir funcionando. También sería necesario implementar respaldos automáticos y mecanismos de recuperación.

12. Conclusiones

La práctica permitió implementar una base de datos distribuida simulada sobre tres contenedores PostgreSQL. Se aplicó fragmentación horizontal en la tabla `pedidos`, fragmentación vertical en la tabla `clientes` y fragmentación mixta combinando criterios de ciudad y sensibilidad de datos.

Los resultados obtenidos demostraron que la fragmentación fue correcta, ya que la vista `pedidos_global` devolvió los 8 pedidos originales y la vista `clientes_global` reconstruyó los 6 clientes completos. Además, la consulta de disjunción devolvió cero filas, lo que confirmó que no existieron registros duplicados.

Finalmente, se concluye que las bases de datos distribuidas pueden mejorar la organización, seguridad y escalabilidad de un sistema, siempre que el diseño sea cuidadosamente verificado.

Referencias

- [1] A. A. Amer, “On K-means clustering-based approach for DDBSs design,” *Journal of Big Data*, vol. 7, no. 31, 2020. doi: 10.1186/s40537-020-00306-9.
- [2] Y.-F. Ge, M. Orlowska, J. Cao, H. Wang, and Y. Zhang, “Knowledge transfer-based distributed differential evolution for dynamic database fragmentation,” *Knowledge-Based Systems*, vol. 229, art. 107325, 2021. doi: 10.1016/j.knosys.2021.107325.
- [3] A. A. Amer, M. H. Mohamed, and K. Al-Asri, “ASGOP: An aggregated similarity-based greedy-oriented approach for relational DDBSs design,” *Heliyon*, vol. 6, no. 1, art. e03172, 2020. doi: 10.1016/j.heliyon.2020.e03172.
- [4] Y.-F. Ge, M. Orlowska, J. Cao, H. Wang, and Y. Zhang, “MDDE: Multitasking distributed differential evolution for privacy-preserving database fragmentation,” *The VLDB Journal*, vol. 31, no. 5, pp. 957–975, 2022. doi: 10.1007/s00778-021-00718-w.